

Distributed Transactions Without a Shared DB

How microservices coordinate multi-step operations across service boundaries

Monolith Had It Easy

✓ Monolith

```
BEGIN TRANSACTION
  reduce stock
  create order
  charge payment
COMMIT
← all or nothing
```

If anything fails → full rollback. Simple.

✗ Microservices

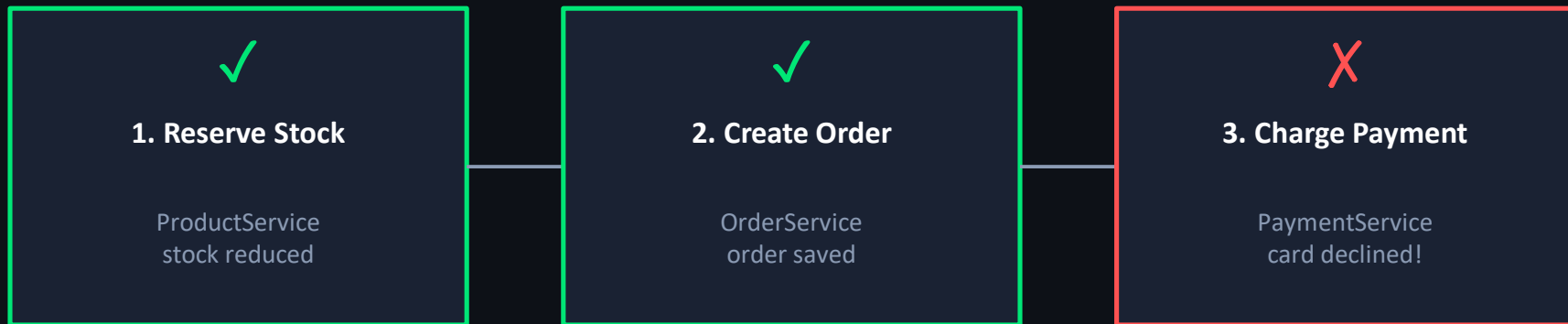
Product Service
products.db

Order Service
orders.db

Payment Service
payments.db

*No shared transaction.
Each service owns its own DB.*

What Happens When a Step Fails?



⚠ **Without coordination, the system is left in an inconsistent state:**

📦 Who un-reserves the stock?

📁 Who cancels the order?

💳 Is the customer charged?

This is exactly the problem Saga solves.

What Is a Saga?

A Saga is a sequence of local transactions. Each step publishes an event or message to trigger the next step. If a step fails, compensating transactions undo the previous steps.

Local Transaction

Each service does its own DB operation — no distributed lock needed.

Compensating Transaction

The 'undo' operation. If step 3 fails, step 2 and 1 must compensate.

No Rollback

Unlike a DB transaction, you can't 'undo' — you must explicitly reverse each step.

Two Flavors

Choreography

Services react to each other's events.
No central coordinator.

OrderService

→ event →

ProductService

→ event →

PaymentService

- ✓ Fully decoupled
 - ✓ No single point of failure
 - ✗ Hard to follow the flow
 - ✗ Harder to debug
-  *Jazz band — everyone reacts*

Orchestration

One coordinator tells each service
what to do, step by step.

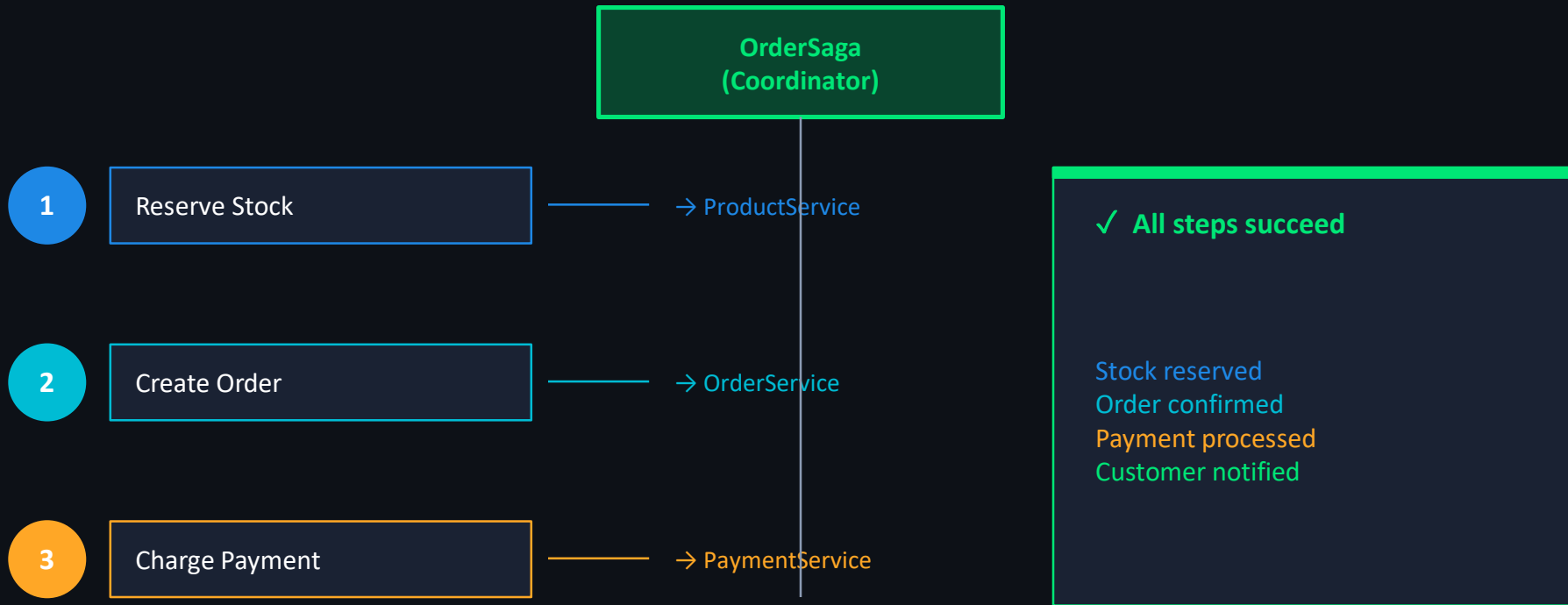
OrderSaga
(Coordinator)

ProductService

PaymentService

- ✓ Easy to read & trace
 - ✓ Clear error handling
 - ✗ Coordinator can be complex
 - ✗ Single class owns the flow
-  *Orchestra — one conductor*

Our Store — Happy Path



Our Store — Payment Fails

✓ 1. Reserve Stock

✓ 2. Create Order

✗ 3. Charge Payment

⚠ Step 3 failed → Saga must compensate all previous steps in reverse order

Compensating Transactions:

↔ 2 Cancel Order

OrderService
Delete the order record

↔ 1 Release Stock

ProductService
Add stock back

End state: stock restored, no order, customer informed of failure — system is consistent again ✓

Choreography — Code Example

Services react to each other's events. No central coordinator.

OrderService + ProductService — emit & react

```
// OrderService: save + emit
class OrderService {
  async placeOrder(order) {
    await db.save(order)
    eventBus.publish('order.created', { orderId: order.id
  })
}

// ProductService: listen & react
class ProductService {
  constructor() {
    eventBus.subscribe('order.created', async (data) => {
      const ok = await this.reserveStock(data.orderId)
      if (ok) eventBus.publish('stock.reserved', data)
      else eventBus.publish('stock.failed', data)
    })
  }
}
```

PaymentService — reacts & compensates

```
class PaymentService {
  constructor() {
    // charge when stock is ready
    eventBus.subscribe('stock.reserved', async (data) => {
      const ok = await this.charge(data.orderId)
      if (!ok) eventBus.publish('payment.failed', data)
    })

    // compensate if stock failed
    eventBus.subscribe('stock.failed', async (data) => {
      await orderService.cancel(data.orderId)
    })
  }
}
```

⚠ No one “owns” the flow. Each service only knows its own event. Decoupled but harder to trace end-to-end.

Orchestration — Code Example

One coordinator class drives every step in order. Easy to read top-to-bottom.

OrderSaga — the coordinator. Reads like a to-do list.

```
class OrderSaga {  
  constructor(productSvc, orderSvc, paymentSvc) {  
    this.productSvc = productSvc  
    this.orderSvc   = orderSvc  
    this.paymentSvc = paymentSvc  
  }  
  
  async run(order) {  
    // step 1  
    const stockOk = await this.productSvc.reserveStock(order.id)  
    if (!stockOk) return this.fail('stock', order)  
    // step 2  
    const orderOk = await this.orderSvc.create(order)  
    if (!orderOk) return this.fail('order', order)  
    // step 3  
    const paid = await this.paymentSvc.charge(order.id)  
    if (!paid)   return this.fail('payment', order)  
    console.log('✓ order complete')  
  }  
  
  async fail(step, order) {  
    // undo in reverse order  
    if (step === 'payment') await this.orderSvc.cancel(order.id)  
    if (step !== 'stock')   await this.productSvc.releaseStock(order.id)  
  }  
}
```

✓ The entire flow is readable in one file. Compensating transactions are in one place — easy to debug.

Key Takeaways

01

The Problem

Microservices can't share a database transaction. A failure in step 3 leaves steps 1 and 2 done with no rollback.

02

What Is a Saga

A sequence of local transactions. Each step publishes an event. Failures trigger compensating transactions.

03

Choreography

Services react to each other's events. Decoupled, but harder to trace. Good for simple flows.

04

Orchestration

A central coordinator drives the flow. Easy to read and debug. Best for complex multi-step operations.

05

Compensating Transactions

The 'undo' of each step. Must be designed upfront — you can't skip them.